# Proyecto - Fase #2 - Analizador sintáctico ascendente

## MiniLang

Fecha de entrega: lunes 20 de abril, 19:30 horas

## 1. Objetivo del Proyecto

Entender las fases de proceso de compilación a través de la creación de un compilador, el estudiante deberá crear de forma incremental el compilador y aplicar todos los conocimientos vistos en clase para la solución del problema.

## 2. Contexto

En esta fase implementarás el analizador sintáctico ascendente para MiniLang, integrándolo con el lexer de la Fase 1. El parser debe validar la estructura del programa, aplicar precedencias y asociatividad en expresiones, y reportar errores sintácticos con recuperación para seguir analizando el resto del archivo.

## 3. Objetivos de la fase

- Definir la gramática de MiniLang y construir el parser (pueden implementar un generador de parsers como yacc/bison/CUP/PLY o equivalente o codificar el suyo propio).
- Integrar lexer + parser: el parser consume tokens del scanner de la Fase 1.
- Resolver precedencia/asociatividad de operadores sin ambigüedades importantes.
- Implementar recuperación de errores para reportar múltiples errores en una sola corrida.

## 4. Alcance del lenguaje (Fase 1)

Se utilizará el subconjunto de un lenguaje de programación estructurado diseñado en fase 1 que incluya:

1. Declaraciones de variables (tipos primitivos como enteros, flotantes, cadenas y booleanos).
2. Expresiones aritméticas con operadores básicos (+, -, *, /).
3. Desigualdades (>, <, =, !=)
4. Estructuras de control (if, else, while).
5. Funciones simples con parámetros.
6. Salida y entrada de datos (Read, Write)

Si implementarán un generador de parser, los estudiantes adecuarán la gramática diseñada al formato requerido por el lexer acompañante (lex, flex, jflex, etc) agregando todas las producciones, las reglas léxicas (Expresiones regulares de la fase I) y la generación de los archivos necesarios para MiniLang.

# 5. Implementación

## 5.1. Precedencia y asociatividad

- Definir precedencias (de menor a mayor): or, and, not, comparaciones, +/-, */%/.
- Asociatividad típica: izquierda para +, -, *, /, %; y derecha para not.
- Evitar conflictos shift/reduce con directivas de precedencia o re-escritura de la gramática de ser necesario.

## 5.2. Manejo y recuperación de errores sintácticos

- Tomar en cuenta que debe reconocer errores léxicos y sintácticos. Los errores deberá reportarlos en pantalla indicando:

  Línea y columna, símbolo que provocó el error, Error: <descripción del error>.
- No se permite abortar en el primer error; se debe intentar continuar hasta EOF.

# 6. Entradas, salidas, set de pruebas y entregables

- El programa debe codificarse en lenguaje de programación a su elección: Java, C#, Python. (Se recomienda C#). El ejecutable o script debe ser llamado **minilang** y debe pedir el archivo de entrada.
- Entrada: archivo con extensión **.mlng** con código MiniLang.
- Si el archivo es sintácticamente correcto: imprimir OK en pantalla. Si hay errores: imprimir la lista de errores sintácticos y continuar hasta EOF.
- Entregables:
  - Programa fuente del analizador sintáctico, que devuelva mensaje de éxito o listado de errores, debidamente documentado internamente
  - Set de casos de prueba utilizados con sus respectivos archivos y resultados esperados (mínimo 2 correctos y 3 con errores).
- Set de pruebas: Se deben entregar al menos 5 pruebas, en las cuales se debe demostrar cómo funcionará su lenguaje, las pruebas deben tener:
  - Prueba 1: Un programa que imprima un “Hola Mundo”.
  - Prueba 2: Un programa que realice una operación aritmética básica.
  - Prueba 3: Un programa que demuestre el uso del input de datos y de acuerdo con la entrada del usuario decidir el flujo del programa a través de un if
  - Prueba 4: Definición de una función que reciba parámetros y devuelva un resultado, la función debe ser llamada desde el programa principal.
  - Prueba 5: Programa que contenga definición de variables, entrada y salida de datos, y que utilice el bucle “while”

## 7. Sobre la entrega

- El proyecto se entregará en un repositorio en GitHub. El código fuente a evaluar será el que esté subido en la sección de entrega del portal. **No se aceptarán cambios en el momento de la calificación.**
- En el repositorio deberá incluir un archivo README donde explique todo el funcionamiento, decisiones de diseño y simplificación de la gramática, y detallando cómo se manejan los errores.
- Aspectos por evaluar
    - Adecuada aplicación de los conocimientos vistos en clase
    - Calidad de la documentación: ortografía, orden, limpieza y que esté completa.
    - Calidad de la solución propuesta: que solucione el problema (que haga lo que requiere el enunciado) en forma eficaz.
    - Funcionalidad del programa: debe cumplir a cabalidad con todos los requerimientos.
    - Evidencia de la creación del programa y dominio de los conceptos utilizados.
    - Crear repositorio en github y agregar el usuario: diana1190.
    - Creatividad.

## 8. Rúbrica de evaluación (100 puntos)

| Criterio | Resultado | Descripción | Ponderación |
|---|---|---|---|
| Solución entregada en el portal | Sí / No | Si no está en el portal no se califican los demás criterios y se obtiene 0 puntos | NA |
| Solución compila | Sí / No | Si la solución no compila no se califican los demás criterios y se obtiene 0 puntos | NA |
| Lectura de archivos correcta | Sí / No | Lee archivos de forma correcta con filtros y no se produce ninguna excepción durante la lectura de archivos | 10 |
| Cobertura de casos de prueba utilizados | Sí / No | Genera los resultados esperados de cada archivo de prueba durante evaluación:<br>1. Reconoce correctamente reservadas, IDs, literales, operadores y puntuación.<br>2. Manejo correcto de pila, NEWLINE significativo y casos borde.<br>3. Acepta todas las construcciones del alcance (if/elif/else, while, def, return, llamadas, print, asignación).<br>4. Errores claros, con ubicación y recuperación sin abortar (cuando aplique).<br>(Se podrá modificar los archivos de prueba durante la calificación) | 35 |
| Defensa de solución | Sí / No | Explica de forma clara y razonada el análisis y | 35 |

| | | | |
|---|---|---|---|
| | | lógica de la solución. Evidencia conocimiento y dominio sobre el código presentado. | |
| Calidad de documentación | Sí / No | README actualizado, instrucciones de ejecución y decisiones técnicas. El código está documentado. | 10 |
| Versionamiento de código | Sí / No | Utiliza Github para las versiones del proyecto | 10 |

Referencias:
https://docs.github.com/en/get-started/start-your-journey/hello-world
https://education.github.com/pack
https://www.gnu.org/software/bison/

**¡Buena suerte!**